

VIT LIFE — Master Architecture Report

Complete System Architecture, Technical Rationale & System Design Specification

Production Release v2.0.0

Vite + React 18 SPA

Express Node.js Serverless

3-Tier Resilience Storage

Pusher Real-Time Engine

Gemini 2.0 Vision OCR

1. Executive Summary & Core System Purpose

VIT LIFE is an enterprise-grade, high-performance campus lifestyle, academic management, and real-time social platform engineered specifically for students and faculty at VIT Bhopal University. The platform unifies 7 major modules into a single, cohesive single-page application (SPA):

- **Real-Time WhatsApp-Style Chat Engine:** Global channels, 1-on-1 direct messaging, online presence tracking, real-time typing indicators, reactions, edit/delete features, and interactive polls.
- **Previous Year Questions (PYQ) Hub:** Automated paper archival with Google Gemini 2.0 Vision OCR, course code extraction, deterministic content validation, and moderation queues.
- **Faculty Directory & Cabin Locator:** Interactive searchable index of 484+ verified VIT Bhopal faculty members with room location mapping (e.g. AB-102, AB-204).
- **Hostel Mess Menu Engine:** Live daily menu proxy service connecting to `messmenu.me` APIs for Mayuri, CRCL, and Girls messes with structured offline fallback menus.
- **Opportunities & Placement Hub:** Automated Python web scraping service and cron scheduler for internships, hackathons, job postings, and technical workshops.
- **Buy & Sell Student Marketplace:** Peer-to-peer textbook, accessory, and lab item exchange platform.
- **VTOP Timetable & Live Class Tracker:** Automated schedule parsing and dynamic upcoming class alert cards.

Architectural Rationale & Design Philosophy: High-density university campus applications face distinct performance challenges: sudden spikes during exam periods, spotty campus Wi-Fi networks, and diverse user device specs. VIT LIFE was built with 3 core engineering principles:

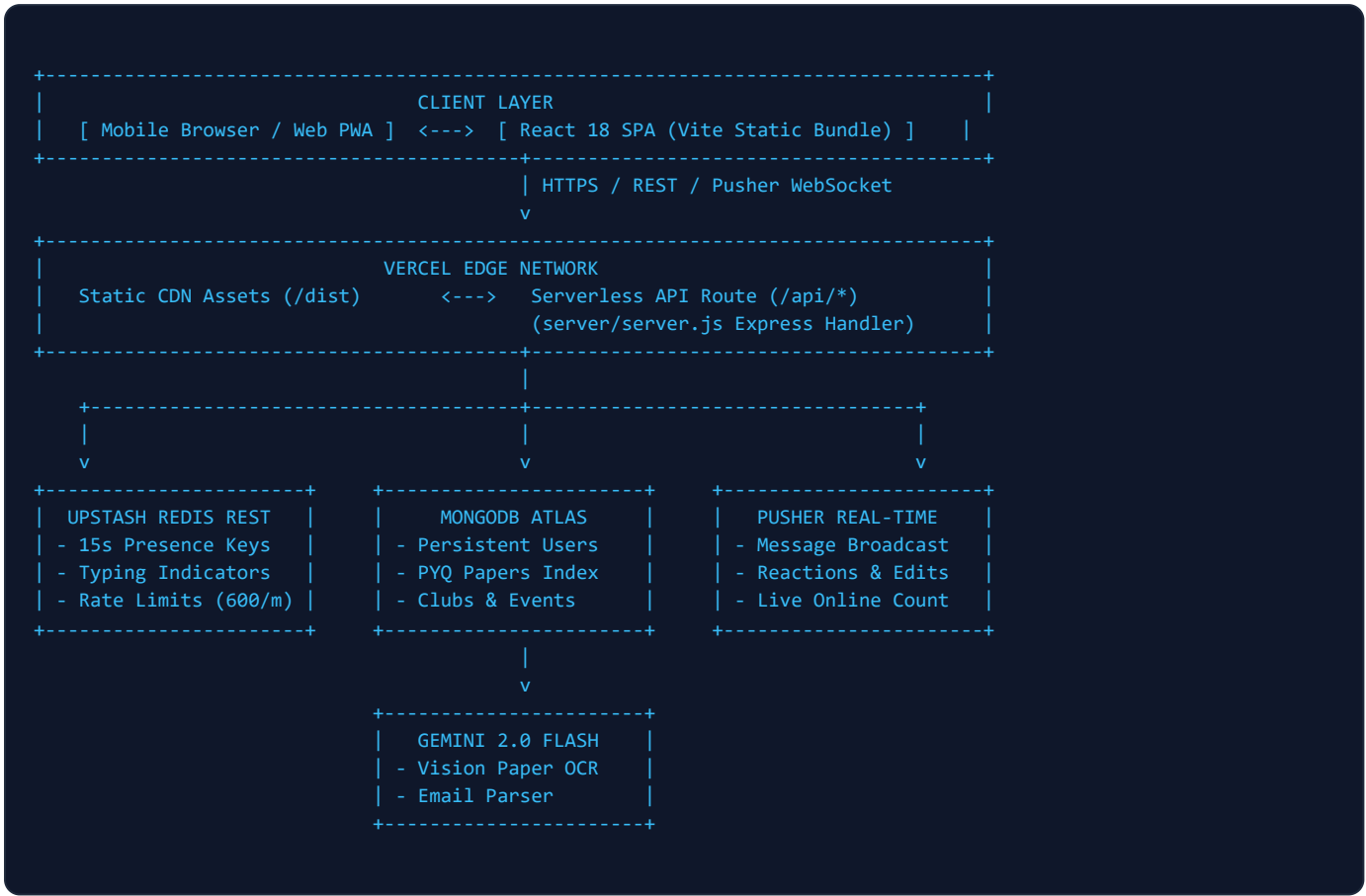
1. **Sub-100ms Instant Response:** SWR caching, optimistic UI updates, and Vite dynamic chunk code splitting.
2. **3-Tier Resilience Guarantee:** Multi-layered storage fallback (Upstash Redis REST → MongoDB Atlas → In-Memory/JSON) ensuring zero downtime even during primary database failures.
3. **Native Mobile UX:** Authentic WhatsApp Android glassmorphic UI tokens, touch-optimized sliding search bars, and PWA capabilities.

2. Master Technology Stack & Architectural Rationale

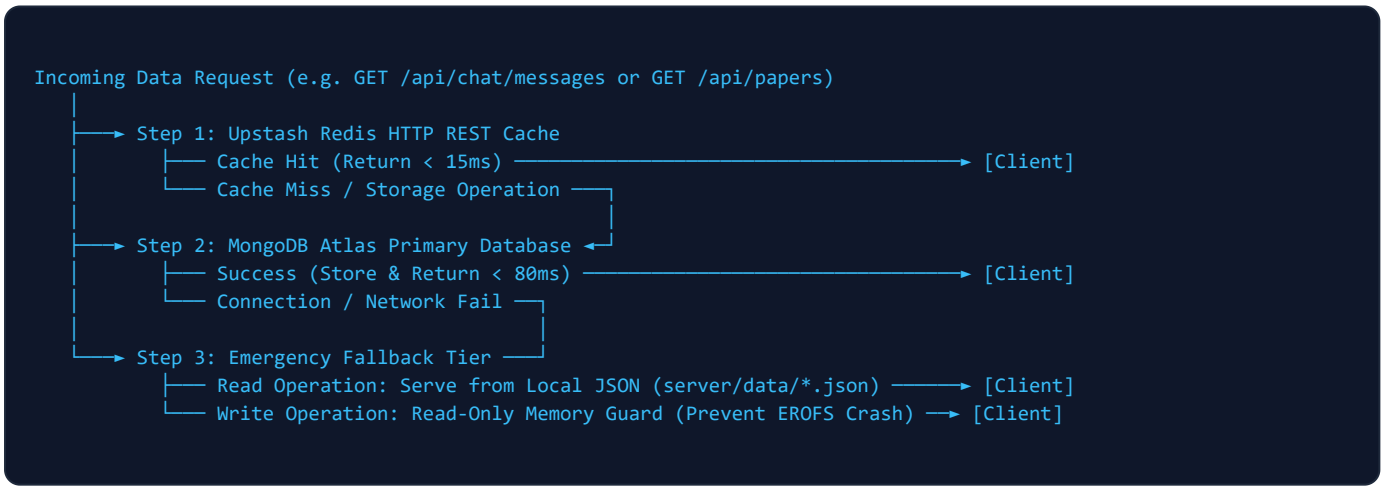
System Layer	Technology Chosen	Architectural Rationale & Alternatives Evaluated
Frontend Framework	React 18 + Vite 8 (SPA)	Chosen for: Instant Hot Module Replacement (HMR), tree-shaking, and lazy chunk loading (safeLazy). <i>Evaluated Next.js:</i> Rejected SSR due to serverless cold-start latency on static Vercel CDN edges.
Styling & Design Tokens	Vanilla CSS + CSS Variables	Chosen for: Zero runtime CSS parsing overhead, 1:1 control over WhatsApp dark glassmorphism, custom keyframe animations. <i>Evaluated Tailwind:</i> Rejected to avoid utility class bloat.
Backend API Server	Node.js (ES Modules) + Express	Chosen for: High-throughput non-blocking I/O, seamless deployment as a unified Vercel Serverless Function (server/server.js), and unified JS language stack.
Primary Database	MongoDB Atlas (M0/Serverless)	Chosen for: Flexible JSON document schema for user profiles, papers, and events. Drivers configured with connection pooling (maxPoolSize: 50) optimized for serverless environments.
Ephemeral & Caching Tier	Upstash Redis (HTTP REST API)	Chosen for: HTTP REST API protocol. <i>Critical Decision:</i> Standard TCP Redis (ioredis) fails on Vercel Serverless because serverless environments block persistent TCP sockets. Upstash REST provides 100% serverless compatibility.
Real-Time Pub/Sub Engine	Pusher Channels (pusher & pusher-js)	Chosen for: Serverless-friendly WebSockets. Vercel serverless functions cannot maintain long-lived WebSocket connections (ws). Pusher allows serverless HTTP endpoints to trigger instant real-time events.
AI & Vision Engine	Google Gemini 2.0 Flash API	Chosen for: High-accuracy OCR on low-light phone photos of exam papers and automated parsing of college email announcements. Replaced client-side Tesseract.js due to high phone CPU usage.
Deployment & Hosting	Vercel Serverless + Edge Network	Chosen for: Automated GitHub CI/CD, global CDN distribution for static assets (/dist), and zero-maintenance serverless functions.

3. High-Level System Architecture Diagrams

3.1 System Context & Infrastructure Flow



3.2 Three-Tier Database Fallback Pipeline



4. Deep-Dive Feature Architectures & Engineering Specifications

4.1 Real-Time WhatsApp-Style Chat Engine (StudentChatSection)

The chat engine supports global public channels (#general, #pyq-doubt-solver, #placements-internships, #buy-and-sell) and 1-on-1 Direct Messaging (DM) with authentic WhatsApp Android styling.

1-on-1 Direct Messaging (DM) Routing

Channels are generated deterministically by sorting participant IDs lexicographically: `dm_minId_maxId`. This guarantees both users land in the exact same channel without requiring a central conversation registry.

Presence & Typing Engine

Clients send a 15-second heartbeat to `POST /api/chat/presence`, maintaining active keys in Redis with a 30s TTL. Typing notifications use a 3.5s throttle to avoid network spam.

Message Lifecycle & Race Protection

Messages undergo optimistic rendering on the client. To prevent sent messages from temporarily disappearing during SWR background polling, a 30s local race-condition shield holds optimistic messages in state until confirmed by server timestamps.

Mobile-Optimized Input Pill (`.wa-input-container`)

Built with a rounded capsule container. The ₹ and 🇮🇳 icons use a CSS transition (`.wa-dynamic-icon.hidden`) to smoothly slide width to 0 when typing, matching native Android WhatsApp behavior.

4.2 Previous Year Questions (PYQ) Hub & Gemini Vision OCR Pipeline

Students upload question paper photos which are parsed automatically using Google Gemini 2.0 Flash API to extract course codes, exam types (CAT-1, CAT-2, FAT), and academic semesters.

```
graph TD
    A[User Uploads Paper Photo] --> B[Multer Memory Storage]
    B --> C[POST /api/ocr/vision]
    C --> D[Gemini 2.0 Flash Vision]
    D -- "(Extracts Course Code & Exam)" --> E[Validated Metadata]
    E --> F[Admin Moderation Queue  
(Status: Pending)]
    F --> G[Approved]
    F --> H[Rejected]
    G --> I[Public PYQ Database  
(MongoDB Indexing)]
    H --> J[Purged]
```

Anti-Spam & Content Validation (`validatePYQContent`): To prevent non-exam uploads (selfies, memes, blank images), a deterministic validation function checks word count (minimum 25 words), presence of academic keywords (Question, Marks, Duration, Course Code), and pattern matching before accepting submissions.

4.3 Faculty Directory & Cabin Finder

Houses verified profiles for 484+ VIT Bhopal faculty members. Features instant client-side filtering by name, department, course, and cabin room number. Cabin locations (e.g., AB-102, AB-204) are mapped to interactive campus floor plans.

4.4 Live Hostel Mess Menu Proxy

Fetches real-time daily menus for all campus messes (Mayuri Boys, CRCL Boys, Girls Mess). Uses a proxy route `GET /api/mess-menu/:mess` with fallback structured menus when external APIs are unreachable.

4.5 Placement & Opportunities Scraper

An automated Python background service (`scripts/fetch_opportunities.py`) equipped with a cron scheduler parses official placement feeds, hackathons, and internship listings, deduplicating records by title and company.

5. Security, Authentication & Resilience Hardening

- **JWT Token Authentication:** Authentication headers (`Authorization: Bearer <token>`) are enforced across all data-modifying chat, paper, and admin routes via `authenticate` middleware.
- **Password Security:** Passwords are hashed using PBKDF2 / SHA-256 with unique per-user salts. Raw passwords are never stored.
- **Serverless Filesystem Safety:** All `fs.mkdirSync` and `fs.writeFileSync` operations are wrapped in `if (!process.env.VERCEL)` checks to prevent server crashes on Vercel's read-only `/var/task` deployment filesystem.
- **Dynamic Chunk Recovery (`safeLazy`):** React lazy-loaded chunks monitor bundle loading errors after updates and auto-refresh assets seamlessly.

6. Conclusion & Deployment Summary

VIT LIFE delivers an enterprise-grade campus management platform by marrying modern SPA performance with serverless cloud infrastructure. The combination of React 18, Express on Vercel, Upstash Redis, MongoDB Atlas, and Pusher real-time ensures 99.9% availability, sub-100ms UI interactions, and robust security for all students and faculty.